

Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen¹

Department of Physics, University of Oslo, Norway¹

February 26, 2025

© 1999-2025, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plans for the week of February 24-28, Solving quantum mechanical problems

1. Repetition from last week on gates, measurements and one-qubit systems
2. Introducing the Variational Quantum Eigensolver (VQE) and discussion of project 1

Readings

1. For the discussion of one-qubit, two-qubit and other gates, sections 2.6-2.11 and 3.1-3.4 of Hundt's book **Quantum Computing for Programmers**, contain most of the relevant information. We will repeat some of these properties during the first part of the lecture.
2. The VQE algorithm is discussed in Hundt's section 6.11
3. See also the VQE review article by Tilly et al.

States, gates and measurements, reminder from preview lectures

Mathematically, quantum gates are a series of unitary operators in the operator space defined by our Hamiltonian $\mathcal{H}$ and operators $\mathcal{O}$ which evolve a given initial state. The unitary nature preserves the norm of the state vector, ensuring the probabilities sum to unity. Since not all gates correspond to an observable, they are not necessarily hermitian.

Single qubit gates

The Pauli matrices (and gate operations following therefrom) are defined as

$$X \equiv \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y \equiv \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z \equiv \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Pauli- X gate

The Pauli- X gate is also known as the **NOT** gate, which flips the state of the qubit.

$$X|0\rangle = |1\rangle,$$

$$X|1\rangle = |0\rangle.$$

The Pauli- Y gate flips the bit and multiplies the phase by i .

$$Y|0\rangle = i|1\rangle,$$

$$Y|1\rangle = -i|0\rangle.$$

The Pauli- Z gate multiplies only the phase of $|1\rangle$ by -1 .

$$Z|0\rangle = |0\rangle,$$

$$Z|1\rangle = -|1\rangle.$$

Hadamard gate

The Hadamard gate is defined as

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

It creates a superposition of the $|0\rangle$ and $|1\rangle$ states.

$$H|0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle), \quad (1)$$

$$H|1\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle). \quad (2)$$

Note that we will use H as symbol for the Hadamard gate while we will reserve the notation $\mathcal{H}$ for a given Hamiltonian.

Phase Gates

The phase gate is usually denoted as S and is defined as

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}.$$

It multiplies only the phase of the $|1\rangle$ state by i .

$$S|0\rangle = |0\rangle,$$

$$S|1\rangle = i|1\rangle.$$

The inverse of the S -gate

The inverse

$$S^\dagger = \begin{pmatrix} 1 & 0 \\ 0 & -i \end{pmatrix}$$

is known as the $S^\dagger$ gate which applies an i phase shift to $|1\rangle$.

$$S^\dagger|0\rangle = |0\rangle,$$

$$S^\dagger|1\rangle = -i|1\rangle.$$

Two-qubit gates

The CNOT gate is a two-qubit gate which acts on two qubits, a control qubit and a target qubit. The CNOT gate is defined as

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

It is often used to perform linear entanglement on qubits.

$$\text{CNOT}|00\rangle = |00\rangle,$$

$$\text{CNOT}|01\rangle = |01\rangle,$$

$$\text{CNOT}|10\rangle = |11\rangle,$$

$$\text{CNOT}|11\rangle = |10\rangle.$$

The SWAP gate

The SWAP gate is a two-qubit gate which swaps the state of two qubits. It is defined as

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

$$\text{SWAP}|00\rangle = |00\rangle,$$

$$\text{SWAP}|01\rangle = |10\rangle,$$

$$\text{SWAP}|10\rangle = |01\rangle,$$

$$\text{SWAP}|11\rangle = |11\rangle.$$

Pauli Strings

A Pauli string, such as $XIYZ$ is a tensor product of Pauli matrices acting on different qubits. The Pauli string $XIYZ$ is defined as (from qubit one to qubit four, from left to right)

$$XIYZ \equiv X_0 \otimes I_1 \otimes Y_2 \otimes Z_3.$$

Hamiltonians are often rewritten or decomposed in terms of Pauli string as they can be easily implemented on quantum computers.

Variational Quantum Eigensolver

One initial algorithm to estimate the eigenenergies of a quantum Hamiltonian was [quantum phase estimation](#). In it, one encodes the eigenenergies, one binary bit at a time (up to n bits), into the complex phases of the quantum states of the Hilbert space for n qubits. It does this by applying powers of controlled unitary evolution operators to a quantum state that can be expanded in terms of the Hamiltonian's eigenvectors of interest. The eigenenergies are encoded into the complex phases in such a way that taking the inverse quantum Fourier transformation (see Hundt sections 6.1-6.2) of the states into which the eigen-energies are encoded results in a measurement probability distribution that has peaks around the bit strings that represent a binary fraction which corresponds to the eigen-energies of the quantum state acted upon by the controlled unitary operators.

The VQE

While quantum phase estimation (QPE) is provably efficient, non-hybrid, and non-variational, the number of qubits and length of circuits required is too great for our NISQ era quantum computers. Thus, QPE is only efficiently applicable to large, fault-tolerant quantum computers that likely won't exist in the near, but the far future.

Therefore, a different algorithm for finding the eigen-energies of a quantum Hamiltonian was put forth in 2014 called the variational quantum eigensolver, commonly referred to as **VQE**. The algorithm is hybrid, meaning that it requires the use of both a quantum computer and a classical computer. It is also variational, meaning that it relies, ultimately, on solving an optimization problem by varying parameters and thus is not as deterministic as QPE. The variational quantum eigensolver is based on the variational principle:

Rayleigh-Ritz variational principle

Out starting point is the Rayleigh-Ritz variational principle states that for a given Hamiltonian H , the expectation value of a trial state or just ansatz $|\psi\rangle$ puts a lower bound on the ground state energy E_0 .

$$\frac{\langle\psi|\mathcal{H}|\psi\rangle}{\langle\psi|\psi\rangle} \geq E_0.$$

The ansatz

The ansatz is typically chosen to be a parameterized superposition of basis states that can be varied to improve the energy estimate, $|\psi\rangle \equiv |\psi(\boldsymbol{\theta})\rangle$ where $\boldsymbol{\theta} = (\theta_1, \dots, \theta_M)$ are the M optimization parameters.

Expectation value of Hamiltonian and the variational principle

The expectation value of a Hamiltonian $\mathcal{H}$ in a state $|\psi(\theta)\rangle$ parameterized by a set of angles θ , is always greater than or equal to the minimum eigen-energy E_0 . To see this, let $|n\rangle$ be the eigenstates of $\mathcal{H}$, that is

$$\mathcal{H}|n\rangle = E_n|n\rangle.$$

Expanding in the eigenstates

We can then expand our state $|\psi(\theta)\rangle$ in terms of the eigenstates

$$|\psi(\theta)\rangle = \sum_n c_n |n\rangle,$$

and insert this in the expression for the expectation value (note that we drop the denominator in the Rayleigh-Ritz ratio)

$$\langle\psi(\theta)|\mathcal{H}|\psi(\theta)\rangle = \sum_{nm} c_m^* c_n \langle m|\mathcal{H}|n\rangle = \sum_{nm} c_m^* c_n E_n \langle m|n\rangle = \sum_{nm} \delta_{nm} c_m^* c_n E_n$$

which implies that we can minimize over the set of angles θ and arrive at the ground state energy E_0

$$\min_{\theta} \langle\psi(\theta)|\mathcal{H}|\psi(\theta)\rangle = E_0.$$

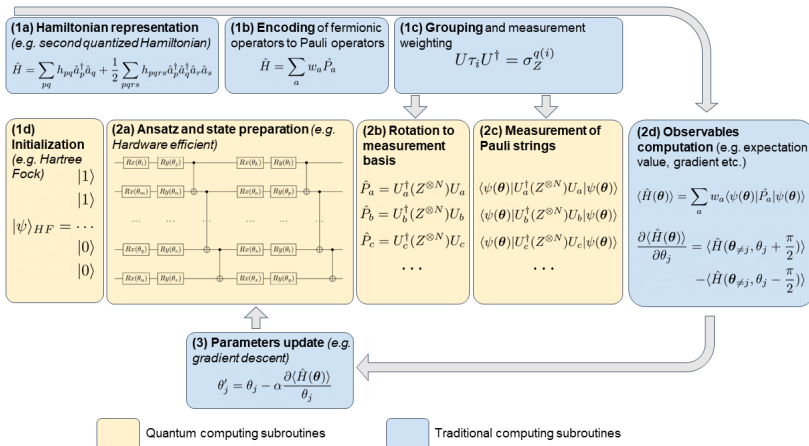
Basic steps of the VQE algorithm

Using this fact, the VQE algorithm can be broken down into the following steps

1. Prepare the variational state $|\psi(\theta)\rangle$ on a quantum computer.
2. Measure this circuit in various bases and send these measurements to a classical computer
3. The classical computer post-processes the measurement data to compute the expectation value $\langle\psi(\theta)|\mathcal{H}|\psi(\theta)\rangle$
4. The classical computer varies the parameters θ according to a classical minimization algorithm and sends them back to the quantum computer which runs step 1 again.

This loop continues until the classical optimization algorithm terminates which results in a set of angles $\theta_{\min}$ that characterize the ground state $|\phi(\theta_{\min})\rangle$ and an estimate for the ground state energy $\langle\psi(\theta_{\min})|\mathcal{H}|\psi(\theta_{\min})\rangle$.

VQE overview



Rotations

To have any flexibility in the ansatz $|\psi\rangle$, we need to allow for a given parametrization. The most common approach is to employ the so-called rotation operations given by R_x , R_z and R_y , where we apply chained operations of rotating around the various axes by $\theta = (\theta_1, \dots, \theta_Q)$ of the Bloch sphere and CNOT operations. Applications of say the y -rotation specifically ensures that our coefficients always remain real, which often is satisfactory when dealing with many-body systems.

Measurements and more

After the ansatz has been constructed, the Hamiltonian must be applied. The Hamiltonian must be written in terms of Pauli strings in order to perform measurements properly.

To obtain the expectation value of the ground state energy, one can measure the expectation value of each Pauli string,

$$E(\boldsymbol{\theta}) = \sum_i w_i \langle \psi(\boldsymbol{\theta}) | P_i | \psi(\boldsymbol{\theta}) \rangle \equiv \sum_i w_i f_i,$$

where f_i is the expectation value of the Pauli string i .

Collecting data

This is estimated statistically by considering measurements in the appropriate basis of the operator in the Pauli string.

With N_0 and N_1 as the number of 0 and 1 measurements respectively, we can estimate f_i since

$$f_i = \lim_{N \rightarrow \infty} \frac{N_0 - N_1}{N},$$

where N as the number of shots (measurements).

Each Pauli string requires its own circuit, where multiple measurements of each string are required. Adding the results together with the corresponding weights, the ground state energy can be estimated. To optimize with respect to θ , a classical optimizer is often applied.

Ansatzes

Every possible qubit wavefunction $|\psi\rangle$ can be presented as a vector:

$$|\psi\rangle = \begin{pmatrix} \cos(\theta/2) \\ e^{i\varphi} \cdot \sin(\theta/2) \end{pmatrix},$$

where the numbers θ and φ define a point on the unit three-dimensional sphere, the so-called Bloch sphere.

For a random one qubit Hamiltonian, a *good* quantum state preparation circuit should be able to generate all possible states on the Bloch sphere.

Preparing the states

Before quantum state preparation, our qubit is in the $|0\rangle$ state. This corresponds to the vertical position of the vector in the Bloch sphere. In order to generate any possible $|\psi\rangle$ we will apply $R_x(t_1)$ and $R_y(t_2)$ gates on the $|0\rangle$ initial state

$$R_y(\phi)R_x(\theta)|0\rangle = |\psi\rangle.$$

The rotation $R_x(\theta)$ corresponds to the rotation in the Bloch sphere around the x -axis and $R_y(\phi)$ the rotation around the y -axis.

Rotations used

These two gates with there parameters (θ and ϕ) will generate for us the trial (ansatz) wavefunctions. The two parameters will be in control of the Classical Computer and its optimization model.

Implementing using qiskit

```
import numpy as np
from random import random
from qiskit import *
def quantum_state_preparation(circuit, parameters):
    q = circuit.qregs[0] # q is the quantum register where the info ab
    circuit.rx(parameters[0], q[0]) # q[0] is our one and only qubit X
    circuit.ry(parameters[1], q[0])
    return circuit
```

Expectation values

To execute the second step of VQE, we need to understand how expectation values of operators can be estimated via quantum computers by post-processing measurements of quantum circuits in different basis sets. To rotate bases, one uses the basis rotator R_σ which is defined for each Pauli gate σ to be (using the Hadamard rotation H and Phase rotation S) for a Pauli- X matrix

$$X = R_\sigma Z R_\sigma = HZH$$

for a Pauli- Y matrix

$$Y = R_\sigma Z R_\sigma = HS^\dagger ZHS,$$

and

$$Z = R_\sigma Z R_\sigma = IZI = Z.$$

Measurements of eigenvalues of the Pauli operators

We can show that these rotations allow us to measure the eigenvalues of the Pauli operators. The eigenvectors of the Pauli X gate are

$$|\pm\rangle = \frac{|0\rangle \pm |1\rangle}{\sqrt{2}},$$

with eigenvalues ± 1 . Acting on the eigenstates with the rotation gives

$$H|+\rangle = +1|0\rangle,$$

and

$$H|-\rangle = -1|1\rangle.$$

Single-qubit states

Any single-qubit state can be written as a linear combination of these eigenvectors,

$$|\psi\rangle = \alpha|+\rangle + \beta|-\rangle.$$

We then have the following expectation value for the Pauli X operator

$$\langle X \rangle = \langle \psi | X | \psi \rangle = |\alpha|^2 - |\beta|^2.$$

However, we can only measure the qubits in the computational basis. Applying the rotation to our state gives

$$H|\psi\rangle = \alpha|0\rangle - \beta|1\rangle.$$

Interpretations

This tells us that we are able to estimate $|\alpha|^2$ and $|\beta|^2$ (and hence the expectation value of the Pauli X operator) by using a rotation and measure the resulting state in the computational basis. We can show this for the Pauli Z and Pauli Y similarly.

Reminder on rotations

Note the following identity of the basis rotator

$$R_{\sigma}^{\dagger} Z R_{\sigma} = \sigma,$$

which follows from the fact that $HZH = X$ and $SXS^{\dagger} = Y$.

Arbitrary Pauli gate

With this, we see that the expectation value of an arbitrary Pauli-gate σ in the state $|\psi\rangle$ can be expressed as a linear combination of probabilities

$$\begin{aligned} E_{\psi}(\sigma) &= \langle\psi|\sigma|\psi\rangle \\ &= \langle\psi|R_{\sigma}^{\dagger}ZR_{\sigma}|\psi\rangle = \langle\phi|Z|\phi\rangle \\ &= \langle\phi|\left(\sum_{x\in\{0,1\}}(-1)^x|x\rangle\langle x|\right)|\phi\rangle \\ &= \sum_{x\in\{0,1\}}(-1)^x|\langle x|\phi\rangle|^2 \\ &= \sum_{x\in\{0,1\}}(-1)^xP(|\phi\rangle\rightarrow|x\rangle), \end{aligned}$$

where $|\phi\rangle = |R_{\sigma}\psi\rangle$ and $P(|\phi\rangle\rightarrow|x\rangle)$ is the probability that the state $|\phi\rangle$ collapses to the state $|x\rangle$ when measured.

Arbitrary string of Pauli operators

This can be extended to any arbitrary Pauli string: consider the string of Pauli operators $P = \bigotimes_{p \in Q} \sigma_p$ which acts non-trivially on the set of qubits Q which is a subset of the total set of n qubits in the system. Then

$$\begin{aligned} E_\psi(P) &= \langle \psi | \left(\bigotimes_{p \in Q} \sigma_p \right) | \psi \rangle \\ &= \langle \psi | \left(\bigotimes_{p \in Q} \sigma_p \right) \left(\bigotimes_{q \notin Q} I_q \right) | \psi \rangle \\ &= \langle \psi | \left(\bigotimes_{p \in Q} R_{\sigma_p}^\dagger Z_p R_{\sigma_p} \right) \left(\bigotimes_{q \notin Q} I_q \right) | \psi \rangle \\ &= \langle \psi | \left(\bigotimes_{p \in Q} R_{\sigma_p}^\dagger \right) \left(\bigotimes_{p \in Q} Z_p \right) \left(\bigotimes_{q \notin Q} I_q \right) \left(\bigotimes_{p \in Q} R_{\sigma_p} \right) | \psi \rangle \end{aligned}$$

Which gives us

$$\begin{aligned}
 E_\psi(P) &= \langle \phi | \left(\bigotimes_{p \in Q} Z_p \right) \left(\bigotimes_{q \notin Q} I_q \right) | \phi \rangle \\
 &= \langle \phi | \left(\bigotimes_{p \in Q} \sum_{x_p \in \{0,1\}} (-1)^{x_p} |x_p\rangle \langle x_p| \right) \left(\bigotimes_{q \notin Q} \sum_{y_q \in \{0,1\}} |y_q\rangle \langle y_q| \right) | \phi \rangle \\
 &= \langle \phi | \left(\sum_{x \in \{0,1\}^n} (-1)^{\sum_{p \in Q} x_p} |x\rangle \langle x| \right) | \phi \rangle \\
 &= \sum_{x \in \{0,1\}^n} (-1)^{\sum_{p \in Q} x_p} |\langle x | \phi \rangle|^2 \\
 &= \sum_{x \in \{0,1\}^n} (-1)^{\sum_{p \in Q} x_p} P(|\phi\rangle \rightarrow |x\rangle),
 \end{aligned}$$

where $|\phi\rangle = |\bigotimes_{p \in Q} R_{\sigma_p} \psi\rangle$.

Final observables

Finally, because the expectation value is linear

$$E_{\psi} \left(\sum_m \lambda_m P_m \right) = \sum_m \lambda_m E_{\psi}(P_m),$$

one can estimate any observable that can be written as a linear combination of Pauli-string terms.

Measurement

To estimate the probability $P(|\phi\rangle \rightarrow |x\rangle)$ from the previous results, one prepares the state $|\phi\rangle$ on a quantum computer and measures it, and then repeats this process (prepare and measure) several times. The probability $P(|\phi\rangle \rightarrow |x\rangle)$ is estimated to be the number of times that one measures the bit-string x divided by the total number of measurements that one makes; that is

$$P(|\phi\rangle \rightarrow |x\rangle) \approx \sum_{m=1}^M \frac{x_m}{M},$$

where $x_m = 1$ if the result of measurement is x and 0 if the result of measurement is not x .

Law of large numbers aka Bernoulli's theorem

By the law of large numbers the approximation approaches equality as M goes to infinity

$$P(|\phi\rangle \rightarrow |x\rangle) = \lim_{M \rightarrow \infty} \sum_{m=1}^M \frac{x_m}{M}.$$

As we obviously do not have infinite time nor infinite quantum computers (which could be run in parallel), we must truncate our number of measurement M to a finite, but sufficiently large number. More precisely, for precision ϵ , each expectation estimation subroutine within VQE requires $\mathcal{O}(1/\epsilon^2)$ samples from circuits with depth $\mathcal{O}(1)$.

VQE and efficient computations of gradients

We start with a reminder on the VQE method with applications to the one-qubit system discussed last week.

Here we revisit the one-qubit system and develop a VQE code for studying this system using gradient descent as a method to optimize the variational ansatz.

We start with a simple 2×2 Hamiltonian matrix expressed in terms of Pauli X and Z matrices, as discussed in the project text.

Symmetric matrix

We define a symmetric matrix $H \in \mathbb{R}^{2 \times 2}$

$$\mathcal{H} = \begin{bmatrix} \mathcal{H}_{11} & \mathcal{H}_{12} \\ \mathcal{H}_{21} & \mathcal{H}_{22} \end{bmatrix},$$

We let $\mathcal{H} = \mathcal{H}_0 + \mathcal{H}_I$, where

$$\mathcal{H}_0 = \begin{bmatrix} E_1 & 0 \\ 0 & E_2 \end{bmatrix},$$

is a diagonal matrix. Similarly,

$$\mathcal{H}_I = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix},$$

where V_{ij} represent various interaction matrix elements.

Non-interacting solution

We can view H_0 as the non-interacting solution

$$\mathcal{H}_0|0\rangle = E_1|0\rangle,$$

and

$$\mathcal{H}_0|1\rangle = E_2|1\rangle,$$

where we have defined the orthogonal computational one-qubit basis states $|0\rangle$ and $|1\rangle$.

Rewriting with Pauli matrices

We rewrite H (and H_0 and H_I) via Pauli matrices

$$\mathcal{H}_0 = \mathcal{E}I + \Omega\sigma_z, \quad \mathcal{E} = \frac{E_1 + E_2}{2}, \quad \Omega = \frac{E_1 - E_2}{2},$$

and

$$\mathcal{H}_I = cI + \omega_z\sigma_z + \omega_x\sigma_x,$$

with $c = (V_{11} + V_{22})/2$, $\omega_z = (V_{11} - V_{22})/2$ and $\omega_x = V_{12} = V_{21}$.
We let our Hamiltonian depend linearly on a strength parameter λ

$$\mathcal{H} = \mathcal{H}_0 + \lambda\mathcal{H}_I,$$

with $\lambda \in [0, 1]$, where the limits $\lambda = 0$ and $\lambda = 1$ represent the non-interacting (or unperturbed) and fully interacting system, respectively.

Selecting parameters

The model is an eigenvalue problem with only two available states. Here we set the parameters $E_1 = 0$, $E_2 = 4$, $V_{11} = -V_{22} = 3$ and $V_{12} = V_{21} = 0.2$.

The non-interacting solutions represent our computational basis. Pertinent to our choice of parameters, is that at $\lambda \geq 2/3$, the lowest eigenstate is dominated by $|1\rangle$ while the upper is $|0\rangle$. At $\lambda = 1$ the $|0\rangle$ mixing of the lowest eigenvalue is 1% while for $\lambda \leq 2/3$ we have a $|0\rangle$ component of more than 90%. The character of the eigenvectors has therefore been interchanged when passing $z = 2/3$. The value of the parameter V_{12} represents the strength of the coupling between the two states.

Setting up the matrix

This part is best seen using the jupyter-notebook

```
from matplotlib import pyplot as plt
import numpy as np
dim = 2
Hamiltonian = np.zeros((dim,dim))
e0 = 0.0
e1 = 4.0
Xnondiag = 0.20
Xdiag = 3.0
Eigenvalue = np.zeros(dim)
# setting up the Hamiltonian
Hamiltonian[0,0] = Xdiag+e0
Hamiltonian[0,1] = Xnondiag
Hamiltonian[1,0] = Hamiltonian[0,1]
Hamiltonian[1,1] = e1-Xdiag
# diagonalize and obtain eigenvalues, not necessarily sorted
EigValues, EigVectors = np.linalg.eig(Hamiltonian)
permute = EigValues.argsort()
EigValues = EigValues[permute]
# print only the lowest eigenvalue
print(EigValues[0])
```

Now rewrite it in terms of the identity matrix and the Pauli matrix X and Z

```
# Now rewrite it in terms of the identity matrix and the Pauli matrix
X = np.array([[0,1],[1,0]])
Y = np.array([[0,1j],[-1j,0]])
```

Measurements and computational basis

We have seen how to rewrite the above 2×2 eigenvalue problem in terms of a Hamiltonian defined by Pauli $\mathbf{X}$ and $\mathbf{Z}$ matrices, and the identity matrix $\mathbf{I}$. Let us make this Hamiltonian that involves only one qubit somewhat more general

$$\langle \psi | \mathcal{H} | \psi \rangle = a \cdot \langle \psi | \mathbf{I} | \psi \rangle + b \cdot \langle \psi | \mathbf{Z} | \psi \rangle + c \cdot \langle \psi | \mathbf{X} | \psi \rangle + d \cdot \langle \psi | \mathbf{Y} | \psi \rangle .$$

Expectation value of I

For the I operator the expectation value is always unity:

$$\langle \psi | I | \psi \rangle = 1.$$

Its contribution to the overall expectation value is thus given by the constant a .

The Pauli matrices

For rest of the Pauli operators, we make the following remark:
every one qubit quantum state $|\psi\rangle$ can be represented via different sets of basis vectors:

$$|\psi\rangle = c_1^z \cdot |0\rangle + c_2^z \cdot |1\rangle = c_1^x \cdot |+\rangle + c_2^x \cdot |-\rangle = c_1^y \cdot |+i\rangle + c_2^y \cdot |-i\rangle .$$

In more detail

We have

$$\text{Z eigenvectors} \quad |0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

For the other two matrices

$$\text{X eigenvectors} \quad |+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix},$$

$$\text{Y eigenvectors} \quad |+i\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}, \quad |-i\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix}.$$

Analyzing these equations

The first presented eigenvectors for each Pauli operator have eigenvalues equal to $+1$: $Z|0\rangle = +1|0\rangle$, $X|+\rangle = +1|+\rangle$, $Y|+i\rangle = +1|+i\rangle$, respectively.

The second eigenvectors for each Pauli operator have eigenvalues equal to -1 : $Z|1\rangle = -1|1\rangle$, $X|-\rangle = -1|-\rangle$, $Y|-i\rangle = -1|-i\rangle$, respectively

Explicit eigenvalues

Now, let's calculate the expectation values of these Pauli operators:

$$\langle \psi | Z | \psi \rangle = (c_1^{z*} \cdot \langle 0 | + c_2^{z*} \cdot \langle 1 |) Z (c_1^z \cdot | 0 \rangle + c_2^z \cdot | 1 \rangle) = |c_1^z|^2 - |c_2^z|^2,$$

$$\langle \psi | X | \psi \rangle = (c_1^{x*} \cdot \langle + | + c_2^{x*} \cdot \langle - |) X (c_1^x \cdot | + \rangle + c_2^x \cdot | - \rangle) = |c_1^x|^2 - |c_2^x|^2,$$

$$\langle \psi | Y | \psi \rangle = (c_1^{y*} \cdot \langle +i | + c_2^{y*} \cdot \langle -i |) Y (c_1^y \cdot | +i \rangle + c_2^y \cdot | -i \rangle) = |c_1^y|^2 - |c_2^y|^2$$

Computational basis

The above equations require that we can make measurements in the chosen basis sets.

However, this may not be possible. The difficulty comes from the fact that one may have the possibility to measure only in the Z -basis. To solve this difficulty we still do a Z -basis measurement, but, before that, we apply specific operators to the $|\psi\rangle$ state.

Unitary transformation of X

If we use the Hadamard gate

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

we can rewrite

$$X = HZH.$$

The Hadamard gate/matrix is a unitary matrix with the property that $H^2 = I$.

Generalizing

For the one-qubit Hamiltonian we have toyed with till now, we can thus rewrite in an easy way the Hamiltonian so that we can perform measurements using our favorite computational basis.

The transformation of the Pauli-***X*** matrix can be generalized, as we will see in more detail below for the two-qubit Hamiltonian and next week for the Lipkin model, to the following expression

$$\mathcal{P} = \mathbf{U}^\dagger \mathbf{M} \mathbf{U},$$

where $\mathcal{P}$ represents some combination of the Pauli matrices and the identity matrix, $\mathbf{U}$ is a unitary matrix and $\mathbf{M}$ represents the gate/matrix which performs the measurements, often represented by a Pauli ***Z*** gate/matrix.

Implementing the VQE

For a one-qubit system we can reach every point on the Bloch sphere (as discussed earlier) with a rotation about the x -axis and the y -axis.

We can express this mathematically through a new state $|\psi\rangle$

$$|\psi\rangle = R_y(\phi)R_x(\theta)|0\rangle.$$

Multiple ansatzes

We can produce multiple ansatzes for the new state in terms of the angles θ and ϕ . With these ansatzes we can in turn calculate the expectation value of the above Hamiltonian, now rewritten in terms of various Pauli matrices (and thereby gates), that is compute

$$\langle \psi | (c + \mathcal{E})I + (\Omega + \omega_z)\sigma_z + \omega_x\sigma_x | \psi \rangle.$$

Rotations again

We can now set up a series of ansatzes for $|\psi\rangle$ as function of the angles θ and ϕ and find thereafter the variational minimum using for example a gradient descent method.

To do so, we need to remind ourselves about the mathematical expressions for the rotational matrices/operators.

$$R_x(\theta) = \cos \frac{\theta}{2} \mathbf{I} - i \sin \frac{\theta}{2} \boldsymbol{\sigma}_x,$$

and

$$R_y(\phi) = \cos \frac{\phi}{2} \mathbf{I} - i \sin \frac{\phi}{2} \boldsymbol{\sigma}_y.$$

Simple code

```
# define the rotation matrices
# Define angles theta and phi
theta = 0.5*np.pi; phi = 0.2*np.pi
Rx = np.cos(theta*0.5)*I-1j*np.sin(theta*0.5)*X
Ry = np.cos(phi*0.5)*I-1j*np.sin(phi*0.5)*Y
#define basis states
basis0 = np.array([1,0])
basis1 = np.array([0,1])

NewBasis = Ry @ Rx @ basis0
print(NewBasis)
# Compute the expectation value
#Note hermitian conjugation
Energy = NewBasis.conj().T @ Hamiltonian @ NewBasis
print(Energy)
```

Not an impressive results. We set up now a loop over many angles θ and ϕ and compute the energies

```
# define a number of angles
n = 20
angle = np.arange(0,180,10)
n = np.size(angle)
ExpectationValues = np.zeros((n,n))
for i in range (n):
    theta = np.pi*angle[i]/180.0
    Rx = np.cos(theta*0.5)*I-1j*np.sin(theta*0.5)*X
    for j in range (n):
```

Gradient descent and calculations of gradients

In order to optimize the VQE ansatz, we need to compute derivatives with respect to the variational parameters. Here we develop first a simpler approach tailored to the one-qubit case. For this particular case, we have defined an ansatz in terms of the Pauli rotation matrices.

Setting up gradients

These define an arbitrary one-qubit state on the Bloch sphere through the expression

$$|\psi\rangle = |\psi(\theta, \phi)\rangle = R_y(\phi)R_x(\theta)|0\rangle.$$

Each of these rotation matrices can be written in a more general form as

$$R_i(\gamma) = \exp\left(-i\frac{\gamma}{2}\sigma_i\right) = \cos\left(\frac{\gamma}{2}\right)I - i\sin\left(\frac{\gamma}{2}\right)\sigma_i,$$

where σ_i is one of the Pauli matrices $\sigma_{x,y,z}$.

Derivatives

It is easy to see that the derivative with respect to γ is

$$\frac{\partial R_i(\gamma)}{\partial \gamma} = -\frac{\gamma}{2} \sigma_i R_i(\gamma).$$

Derivatives of the expectation value of the Hamiltonian

We can now calculate the derivative of the expectation value of the Hamiltonian in terms of the angles θ and ϕ . We have two derivatives

$$\frac{\partial}{\partial \theta} [\langle \psi(\theta, \phi) | \mathbf{H} | \psi(\theta, \phi) \rangle] = \frac{\partial}{\partial \theta} [\langle \mathbf{H}(\theta, \phi) \rangle] = \langle \psi(\theta, \phi) | \mathbf{H}(-\frac{i}{2} \boldsymbol{\sigma}_x | \psi(\theta, \phi) \rangle +$$

and

$$\frac{\partial}{\partial \phi} [\langle \psi(\theta, \phi) | \mathbf{H} | \psi(\theta, \phi) \rangle] = \frac{\partial}{\partial \phi} [\langle \mathbf{H}(\theta, \phi) \rangle] = \langle \psi(\theta, \phi) | \mathbf{H}(-\frac{i}{2} \boldsymbol{\sigma}_y | \psi(\theta, \phi) \rangle -$$

Two additional expectation values

This means that we have to calculate two additional expectation values in addition to the expectation value of the Hamiltonian itself. If we stay with an ansatz for the single qubit states given by the above rotation operators, we can, following for example [the article by Maria Schuld et al](#), show that the derivative of the expectation value of the Hamiltonian can be written as (we focus only on a given angle ϕ)

$$\frac{\partial}{\partial \phi} [\langle \mathbf{H}(\phi) \rangle] = \frac{1}{2} \left[\langle \mathbf{H}(\phi + \frac{\pi}{2}) \rangle - \langle \mathbf{H}(\phi - \frac{\pi}{2}) \rangle \right].$$

Rotations again and again

To see this, consider again the definition of the rotation operators. We can write these operators as

$$R_i(\phi) = \exp -i(\phi\sigma_i),$$

with ***sigma***_{*i*}, with σ_i being any of the Pauli matrices X , Y and Z . The latter can be generalized to other unitary matrices as well. The derivative with respect to ϕ gives

$$\frac{\partial R_i(\phi)}{\partial \phi} = -i\sigma_i \exp -i(\phi\sigma_i) = -i\sigma_i R_i(\phi).$$

Bloch sphere math

Our ansatz for a general one-qubit state on the Bloch sphere contains the product of a rotation around the x -axis and the y -axis. In the derivation here we focus only on one angle however. Our ansatz is then given by

$$|\psi\rangle = R_i(\phi)|0\rangle,$$

and the expectation value of our Hamiltonian is

$$\langle\psi|\hat{H}|\psi\rangle = \langle 0|R_i(\phi)^\dagger\hat{H}R_i(\phi)|0\rangle.$$

Derivatives

Our derivative with respect to the angle ϕ has a similar structure, that is

$$\frac{\partial}{\partial \phi} [\langle \psi(\theta, \phi) | \mathbf{H} | \psi(\theta, \phi) \rangle] = \langle \psi(\theta, \phi) | \mathbf{H} (-\frac{i}{2} \sigma_y | \psi(\theta, \phi) \rangle + \text{h.c.}$$

Rewriting

In order to rewrite the equation of the derivative, the following relation is useful

$$\langle \psi | \hat{A}^\dagger \hat{B} \hat{C} | \psi \rangle = \frac{1}{2} \left[\langle \psi | (\hat{A} + \hat{C})^\dagger \hat{B} (\mathbf{A} + \hat{C}) | \psi \rangle - \langle \psi | (\hat{A} - \hat{C})^\dagger \hat{B} (\mathbf{A} - \hat{C}) | \psi \rangle \right]$$

where $\hat{A}$, $\hat{B}$ and $\hat{C}$ are arbitrary hermitian operators.

Final manipulations

If we identify these operators as $\hat{A} = I$, with I being the unit operator, $\hat{B} = \hat{H}$ our Hamiltonian, and $\hat{C} = -i\sigma_i/2$, we obtain the following expression for the expectation value of the derivative (excluding the hermitian conjugate)

$$\langle \psi | I^\dagger \hat{H} (-\frac{i}{2} \sigma_i | \psi \rangle = \frac{1}{2} \left[\langle \psi | (I - \frac{i}{2} \sigma_i)^\dagger \hat{H} (I - \frac{i}{2} \sigma_i) | \psi \rangle - \langle \psi | (I + \frac{i}{2} \sigma_i)^\dagger \hat{H} (I + \frac{i}{2} \sigma_i) | \psi \rangle \right]$$

The expressions to implement

If we then use that the rotation matrices can be rewritten as

$$R_i(\phi) = \exp\left(-i\frac{\phi}{2}\sigma_i\right) = \cos\left(\frac{\phi}{2}\right)\mathbf{I} - i\sin\left(\frac{\phi}{2}\right)\sigma_i,$$

we see that if we set the angle to $\phi = \pi/2$, we have

$$R_i\left(\frac{\pi}{2}\right) = \cos\left(\frac{\pi}{4}\right)\mathbf{I} - i\sin\left(\frac{\pi}{4}\right)\sigma_i = \frac{1}{\sqrt{2}}\left(\mathbf{I} - i\sigma_i\right).$$

Final expression

This means that we can write

$$\langle \psi | \hat{H} | -\frac{i}{2} \sigma_i | \psi \rangle = \frac{1}{2} \left[\langle \psi | R_i(\frac{\pi}{2})^\dagger \hat{H} R_i(\frac{\pi}{2}) | \psi \rangle - \langle \psi | R_i(-\frac{\pi}{2})^\dagger \hat{H} R_i(-\frac{\pi}{2}) | \psi \rangle \right]$$

Basics of gradient descent and stochastic gradient descent

In order to implement the above equations, we need to remind the reader about basic elements of various optimization approaches. Our main focus here will be various gradient descent approaches and quasi-Newton methods like Broyden's algorithm and variations thereof.

This material is covered by the lectures from [FYS4411 on gradient optimization](#)

Computing quantum gradients

Let us implement efficient implementations of gradient methods to the derivatives of the Hamiltonian expectation values.

```
from matplotlib import pyplot as plt
import numpy as np
from scipy.optimize import minimize
dim = 2
Hamiltonian = np.zeros((dim,dim))
e0 = 0.0
e1 = 4.0
Xnondiag = 0.20
Xdiag = 3.0
Eigenvalue = np.zeros(dim)
# setting up the Hamiltonian
Hamiltonian[0,0] = Xdiag+e0
Hamiltonian[0,1] = Xnondiag
Hamiltonian[1,0] = Hamiltonian[0,1]
Hamiltonian[1,1] = e1-Xdiag
# diagonalize and obtain eigenvalues, not necessarily sorted
EigValues, EigVectors = np.linalg.eig(Hamiltonian)
permute = EigValues.argsort()
EigValues = EigValues[permute]
# print only the lowest eigenvalue
print(EigValues[0])

# Now rewrite it in terms of the identity matrix and the Pauli matrix
X = np.array([[0,1],[1,0]])
Y = np.array([[0,-1j],[1j,0]])
```


A smarter way of doing this

The above approach means that we are setting up several matrix-matrix and matrix-vector multiplications. Although straight forward it is not the most efficient way of doing this, in particular in case the matrices become large (and sparse). But there are some more important issues.

In a physical realization of these systems we cannot just multiply the state with the Hamiltonian. When performing a measurement we can only measure in one particular direction. For the computational basis states which we have, $|0\rangle$ and $|1\rangle$, we have to measure along the bases of the Pauli matrices and reconstruct the eigenvalues from these measurements.

The code for the one qubit case

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set_theme(font_scale=1.5)
from tqdm import tqdm

sigma_x = np.array([[0, 1], [1, 0]])
sigma_y = np.array([[0, -1j], [1j, 0]])
sigma_z = np.array([[1, 0], [0, -1]])
I = np.eye(2)

def Hamiltonian(lmb):
    E1 = 0
    E2 = 4
    V11 = 3
    V22 = -3
    V12 = 0.2
    V21 = 0.2

    eps = (E1 + E2) / 2
    omega = (E1 - E2) / 2
    c = (V11 + V22) / 2
    omega_z = (V11 - V22) / 2
    omega_x = V12

    H0 = eps * I + omega * sigma_z
    H1 = c * I + omega_z * sigma_z + omega_x * sigma_x
    return H0 + lmb * H1
```

Plans for the week of February March 1

1. Reminder on basics of the VQE method and how to perform measurements for the simpler two-qubit Hamiltonians
2. Introducing the Lipkin model
3. Solving problems with more than one qubit and discussion of project 1
4. Reading suggestion, VQE review article